

KOCAELİ ÜNİVERSİTESİ MÜHENDİSLİK FAKÜLTESİ

**Gömülü Sistem Üzerinde Görüntü İşleme Tabanlı Yapay Zeka
Destekli Dost-Düşman Hedef Tespit ve Takip Sistemi**

**SUDE ÇAKIR
AHMET FATİH BAŞAR**

Bölümü: Elektronik ve Haberleşme Mühendisliği

KOCAELİ, 2025

İÇİNDEKİLER

İÇİNDEKİLER	ii
ŞEKİLLER DİZİNİ.....	iii
1. GİRİŞ	1
2. PROJEDE YAPILAN ÇALIŞMALAR.....	4
2.1 Kavramsal Tasarım.....	4
2.2 İş Zaman Çizelgesi	5
2.3 Projede Gerçekleşen İş Paketleri ve Ara Çıktıları.....	6
2.3.1 Veri Setinin Hazırlanması	7
2.3.2 Yapay Zeka Modelinin Eğitimi.....	8
2.3.3 Bilgisayar Ortamında Model Testleri ve Tespit İşaretleme Algoritmasının Oluşturulması	11
2.3.3.1 Bilgisayar Ortamında Tespit Testleri	12
2.3.3.2 Tespit İşaretleme Algoritmasının Bilgisayar Ortamında Gerçekleştirilmesi ..	13
2.3.4 Raspberry Pi’da Sistem Kurulumu ve YOLO Algoritmasının Uygulanması	14
2.3.5 Raspberry Pi’da Gerçek Zamanlı Tespit İşaretleme Algoritmalarının Uygulanması	19
2.3.5.1 Gerçek Zamanlı Tespit Akışının Genel Mimarisi.....	19
2.3.5.2 Finite-State-Machine (FSM) Tabanlı İşaretleme Sistemi	20
2.3.5.3 Gerçek Zamanlı Tespit ve İşaretleme Çıktıları	20
2.3.5.4 Raspberry Pi Üzerinde Performans (FPS-Gecikme) Analizi	22
3. KAYNAKLAR	25

ŞEKİLLER DİZİNİ

[1] Şekil 1: Kırmızı-Mavi balon hedef tespit blok gösterimi	2
[2] Şekil 2: İş-Zaman Çizelgesi Gantt Şeması.....	5
[3] Şekil 3: V1 sürümüne ait Roboflow veri seti ekranı.....	7
[4] Şekil 4: V2 sürümüne ait Roboflow veri seti ekranı.....	8
[5] Şekil 5: YOLOv8 Eğitim Sürecine Ait Kayıp Fonksiyonu ve Performans Eğrileri.....	10
[6] Şekil 6: Eğitilmiş YOLOv8 Modeline Ait Confusion Matrix Çıktısı.....	10
[7] Şekil 7: YOLOv8 validasyon performans sonuçları.....	11
[8] Şekil 8: Gerçek Zamanlı Webcam Çıktısı.....	12
[9] Şekil 9: Video testlerinden örnek kareler.....	13
[10] Şekil 10: Lock–Impact–Kill algoritmasının bilgisayar ortamında real-time testi.....	14
[11] Şekil 11: Raspberry Pi 5.....	15
[12] Şekil 12: ONNX → NCNN dönüşüm aşamalarını gösteren terminal çıktıları.....	16
[13] Şekil 13: Raspberry Pi 5 üzerinde NCNN tabanlı YOLOv8n modelinin 854×480 çözünürlüğünde tespit performansı ve FPS çıktısı.....	18
[14] Şekil 14: Raspberry Pi 5 Kamera ile NCNN tabanlı YOLOv8n modelinin 854×480 çözünürlüğünde tespit performansı ve FPS çıktısı.....	18
[15] Şekil 15: Raspberry Pi Gerçek Zamanlı Tespit–İşaretleme Algoritması Akış Şeması.....	20
[16] Şekil 16: Raspberry Pi üzerinde gerçek zamanlı tespit – LOCK durumu.....	21
[17] Şekil 17: Raspberry Pi Üzerinde İşaretleme – IMPACT durumu.....	21
[18] Şekil 18: Gerçek zamanlı çalışmada birden fazla hedefin yönetimi.....	22
[19] Şekil 19: Raspberry Pi üzerinde performans analiz görüntüsü 1.....	23
[20] Şekil 20: Raspberry Pi üzerinde performans analiz görüntüsü 2.....	23
[21] Şekil 21: Destroy_locked.csv dosyası.....	24
[22] Şekil 22: Destroy_locked.xml dosyası.....	24

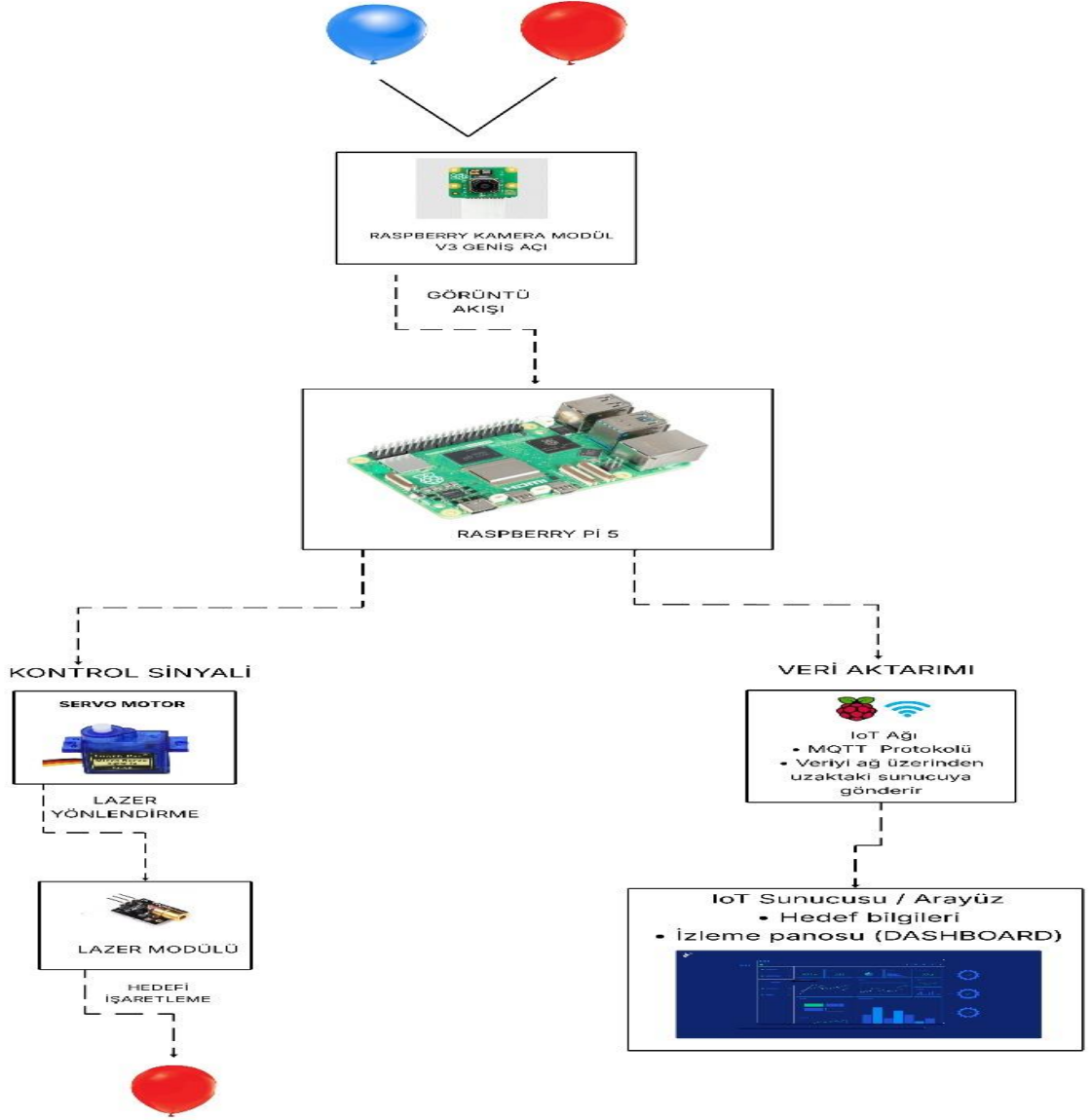
1. GİRİŞ

Günümüzde savunma sistemlerinde hedef tespiti, sınıflandırma ve takip süreçlerinin hızlı, güvenilir ve uzaktan izlenebilir olması büyük önem taşımaktadır. Özellikle otonom gözetleme ve insansız sistemlerde, gerçek zamanlı karar verebilen gömülü yapay zeka çözümlerine olan ihtiyaç hızla artmaktadır. Bu doğrultuda geliştirilen projede, görüntü işleme teknikleri kullanılarak dost–düşman hedef ayrımı yapabilen, tespit edilen düşman hedefi servo kontrollü bir mekanizma ile takip edip merkezleyebilen ve doğrulama sonrası lazer işaretleme simülasyonu gerçekleştirebilen bir gömülü sistem tasarlanması amaçlanmaktadır.

Sistem, kamera modülü üzerinden alınan görüntüyü Raspberry Pi üzerinde çalışan yapay zeka modeliyle analiz etmekte ve tespit edilen nesneleri konum, renk ve sınıf bilgilerine göre değerlendirmektedir. Düşman hedef algılandığında pan–tilt servo mekanizması hedefi görüş hattının merkezine getirmekte ve lazer modülü ile işaretleme yapılmaktadır.

Bu projede geliştirilen sistem yalnızca yerel çalışmayıp, aynı zamanda IoT altyapısına entegre edilerek uzak erişim ve izleme yeteneğine de sahip olacaktır. Tespit edilen hedef türü, konumu, zaman damgası ve sistem durumu gibi bilgiler MQTT tabanlı bir yapıyla buluta aktarılacak; kullanıcılar web veya mobil uyumlu özel olarak tasarlanmış bir arayüz üzerinden sistemi gerçek zamanlı olarak izleyebilecektir. Bu arayüz; hedef tespit geçmişi, anlık görüntü akışı, sistem durumu, servo açılı bilgileri ve lazer aktivasyon kayıtlarını kullanıcıya görsel ve anlaşılır bir formatta sunacaktır.

Bu çalışma; görüntü işleme, gömülü yapay zeka, servo kontrol, IoT mimarisi ve kullanıcı arayüz tasarımı gibi birçok alanı bir araya getirerek kapsamlı bir mühendislik uygulaması ortaya koymaktadır. Projenin uygulama alanları arasında otonom gözetleme sistemleri, güvenlik robotları, insansız araçlar, laboratuvar prototipleri ve savunma teknolojilerinde kullanılan hedef tespit–takip platformları bulunmaktadır.



Şekil 1: Kırmızı-Mavi balon hedef tespit blok gösterimi

Sistemin genel işleyişini göstermekte olup sahnedeki hedeflerin renk temelli yapay zekâ analizi ile ayırt edilmesini temel almaktadır. Diyagramda yer alan mavi ve kırmızı balonlar, proje kapsamında kullanılan dost ve düşman hedef modellerini temsil etmektedir. Mavi balon dost hedef olarak kabul edilirken, kırmızı balon düşman hedef olarak değerlendirilmekte ve sistemin tepkisi yalnızca düşman hedefe karşı tetiklenmektedir.

Sahnedeki balonlar Raspberry Pi Kamera Modülü V3 Geniş Açı tarafından görüntülenmekte ve elde edilen görüntü akışı Raspberry Pi 5'e iletilmektedir. Raspberry Pi üzerinde çalışan yapay zekâ modeli, balonları renk, şekil ve konum bilgileri üzerinden analiz ederek hedef tespiti gerçekleştirmektedir. Bu aşamada sistem, balonun rengine göre dost–düşman sınıflandırmasını gerçek zamanlı olarak yapmaktadır.

Düşman hedef (kırmızı balon) tespit edildiğinde Raspberry Pi, servo motorlara kontrol sinyali göndererek lazer modülünün yönünü hedefe göre ayarlamaktadır. Servo motor, lazeri düşman hedefin merkezine getirecek şekilde konumlandırır ve lazer modülü hedefi işaretler. Böylece sistem, otonom bir hedef tespit–takip ve lazerle işaretleme mekanizması oluşturur.

Ayrıca sistem, yalnızca fiziksel işaretleme yapmayıp tespit edilen dost veya düşman hedef bilgilerini, konum verilerini ve zaman damgalarını IoT altyapısı üzerinden uzak bir sunucuya iletmektedir. Bu amaçla MQTT protokolü kullanılmakta ve veriler kablosuz ağ üzerinden IoT sunucusuna aktarılmaktadır. Kullanıcılar, özel olarak tasarlanmış bir izleme arayüzü (dashboard) üzerinden hedef tespit geçmişini, anlık sistem durumunu, tespit edilen balonun türünü (kırmızı/mavi), konum bilgisini ve sistem aktivitelerini gerçek zamanlı olarak takip edebilmektedir.

Bu yapı sayesinde sistem hem laboratuvar ortamında dost–düşman ayrımı yapabilen otonom bir fiziksel prototip olarak çalışmakta hem de IoT entegrasyonu sayesinde uzaktan izlenebilir bir güvenlik platformu haline gelmektedir.

2. PROJEDE YAPILAN ÇALIŞMALAR

2.1 Kavramsal Tasarım

Projenin ilk aşamasında, dost ve düşman hedeflerinin tanınması için kullanılacak veri tabanının oluşturulmasına başlanmıştır. Bu kapsamda kırmızı ve mavi balonlara ait çeşitli açılardan, farklı ışık koşullarında ve farklı arka planlarda çekilmiş görüntüler toplanmış; bu görüntüler Roboflow platformuna yüklenerek etiketleme ve ön işleme süreçleri uygulanmıştır. Veri kümesinin çeşitliliğinin artırılması ve model eğitime uygun hâle getirilmesi için augmentasyon ve optimizasyon çalışmaları devam etmektedir. Hazırlanan veri kümesi ilerleyen aşamalarda YOLOv8 modelinin eğitiminde kullanılmaktadır.

Bu veri kümesi ile YOLOv8 modeli üzerinde birden fazla eğitim denemesi yapılmış ve ilk test sonuçları incelenmiştir. Farklı epoch, batch size ve veri artırma yöntemleri denenerek model performansının artırılması hedeflenmiştir. Eğitim süreci hâlâ geliştirme aşamasındadır ve performans analizleri doğrultusunda veri kümesinin yeniden düzenlenmesi veya ek görüntülerle desteklenmesi planlanmaktadır. Bu sayede kırmızı ve mavi balonların doğru sınıflandırılmasını sağlayacak daha kararlı bir yapay zekâ modeli oluşturulacaktır.

Elde edilen modeller ilerleyen aşamada Raspberry Pi 5 üzerinde gerçek zamanlı olarak test edilmiştir. Raspberry Pi kamera modülü ile alınan görüntülerin canlı olarak modele aktarılması, modelin sahnedeki balonları tespit edip sınıflandırması ve sistemin kare bazlı performansının ölçülmesi bu aşamada gerçekleştirilmiştir. Gerçek zamanlı testlerde servo motorların hedefe yönelmesi ve lazer modülünün düşman balonu işaretlemesi gibi fiziksel kontrol süreçleri de değerlendirilecektir. Bu aşama hâlen hazırlık sürecinde olup testler raporun ilerleyen dönemlerinde gerçekleştirilecektir.

Son aşamada sistemden elde edilen hedef bilgileri IoT altyapısına aktarılacak ve MQTT protokolü ile uzak bir sunucuya gönderilecektir. Bu kapsamda geliştirilecek dashboard arayüzünde dost ve düşman hedeflerin gerçek zamanlı gösterimi, tespit

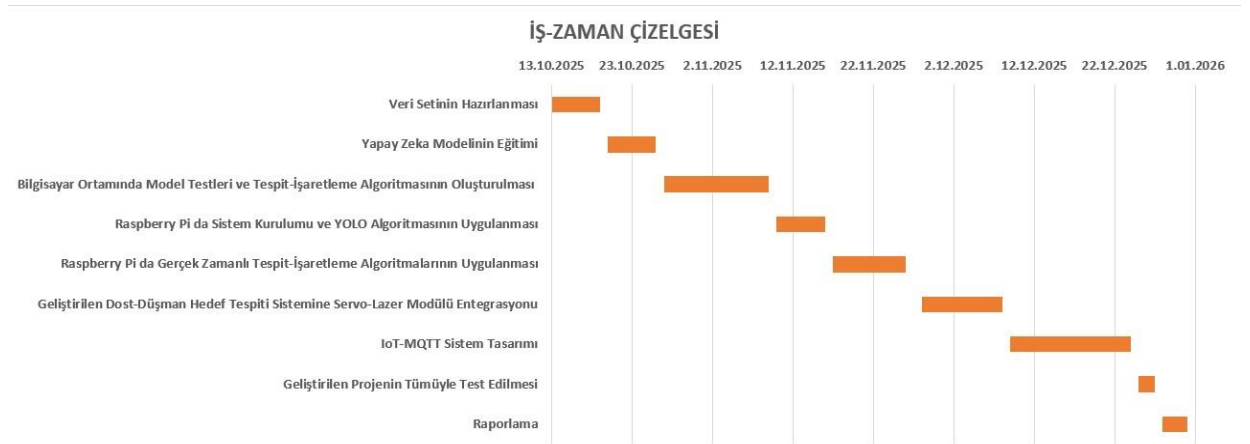
geçmişi ve sistem durumu kullanıcıya sunulacaktır. IoT entegrasyonu şu anda planlama aşamasında olup gerekli yazılım ve sunucu düzenlemeleri tamamlandıktan sonra hayata geçirilecektir. Projenin genel akışı kapsamında veri seti oluşturma, model eğitimi, gerçek zamanlı testler ve IoT aktarımı aşamaları adım adım ilerletilmekte olup dönem sonunda tamamlanması hedeflenmektedir.

2.2 İş-Zaman Çizelgesi

Projenin uygulanma süreci, belirlenen çalışma takvimi doğrultusunda iş paketlerine ayrılarak planlanmıştır. Şekil 2’de gösterilen Gantt şeması, proje boyunca yürütülecek faaliyetlerin zamanlamasını, iş paketlerinin birbirleriyle olan ilişkilerini ve her bir aşamanın tamamlanma süresini göstermektedir.

Bu kapsamda proje; veri setinin hazırlanması, yapay zekâ modelinin eğitilmesi, bilgisayar ortamında model doğrulama testleri, Raspberry Pi üzerinde gerçek zamanlı algoritmaların uygulanması, servo-lazer modülü ile fiziksel işaretleme sisteminin oluşturulması ve IoT tabanlı izleme panosunun tasarlanması gibi temel aşamalardan oluşmaktadır. Son aşamada geliştirilen tüm bileşenlerin birlikte test edilmesi ve raporlanması hedeflenmiştir.

Şekil 2’de sunulan iş-zaman çizelgesi, projenin ara rapor dönemine kadar tamamlanan görevlerle gelecek dönemlerde yürütülecek çalışmaları açıkça göstermekte ve projenin planlanan takvim doğrultusunda ilerlediğini ortaya koymaktadır.



Şekil 2: İş-Zaman Çizelgesi Gantt Şeması

2.3 Projede Gerçekleşen İş Paketleri ve Ara Çıktıları

Projenin başlangıcından 27.11.2025 tarihli ara rapor dönemine kadar yürütülen çalışmalar, belirlenen iş-zaman çizelgesine uygun şekilde ilerlemiş ve kritik iş paketlerinin tamamı planlanan tarihler içerisinde başarıyla gerçekleştirilmiştir. Veri setinin hazırlanması, yapay zekâ modelinin eğitilmesi, bilgisayar ortamında model testleri ve gerçek zamanlı tespit-işaretleme algoritmasının oluşturulması gibi temel aşamalar eksiksiz tamamlanmış; ayrıca modelin Raspberry Pi üzerinde çalıştırılması ve gömülü ortamda ilk tespit denemelerinin yapılmasıyla sistemin bir sonraki faza geçebilmesi için gerekli altyapı hazırlanmıştır. Proje bu aşamada hem plan ile uyumlu ilerlemekte hem de sonraki adımlar olan servo-lazer entegrasyonu ve IoT mimarisine geçiş için teknik birikimi sağlamış bulunmaktadır.

2.3.1 Veri Setinin Hazırlanması

Proje Proje kapsamında kullanılan veri seti tamamen özgün olarak oluşturulmuş olup herhangi bir hazır veya açık kaynak balon veri seti kullanılmamıştır. Veri toplama süreci, kırmızı (düşman) ve mavi (dost) balonların farklı ortam koşullarında, çeşitli açılardan ve farklı ışık seviyelerinde görüntülenmesiyle gerçekleştirilmiştir. Bu süreçte yalnızca proje ekibi tarafından çekilen ham görüntüler kullanılmış, böylece modelin gerçek sahneye uyumluluğunu artıracak yüksek çeşitlilikte bir veri kümesi elde edilmiştir.

Oluşturulan veri setinin ilk sürümü (V1), toplam **2446 adet ham görüntünün** Roboflow platformunda **manuel olarak etiketlenmesiyle** hazırlanmıştır. Bu aşamada her balon örneğinin sınıf bilgisi (red/blue) ve konum bilgisi (bounding box) tek tek işaretlenmiş, veri kalitesini artırmak için Roboflow'un sunduğu augmentasyon teknikleri uygulanmıştır. V1 versiyonundaki veri setine; yatay çevirme, -9° ile $+9^{\circ}$ arası dönüş, -12% ile $+12\%$ arası parlaklık değişimi ve -8% ile $+8\%$ arası pozlama (exposure) değişimi gibi dönüştürme işlemleri uygulanmış ve Roboflow'un sağladığı “2x çoğaltma (augment per example $\times 2$)” yeteneği ile veri seti genişletilmiştir. Bu işlem sonucunda V1 veri seti toplam **4892 görüntüye** ulaşmıştır. Split dağılımı Roboflow tarafından **%82 eğitim (4026 görüntü)**, **%12 doğrulama (577 görüntü)** ve **%6 test (289 görüntü)** olacak şekilde otomatik olarak düzenlenmiştir.

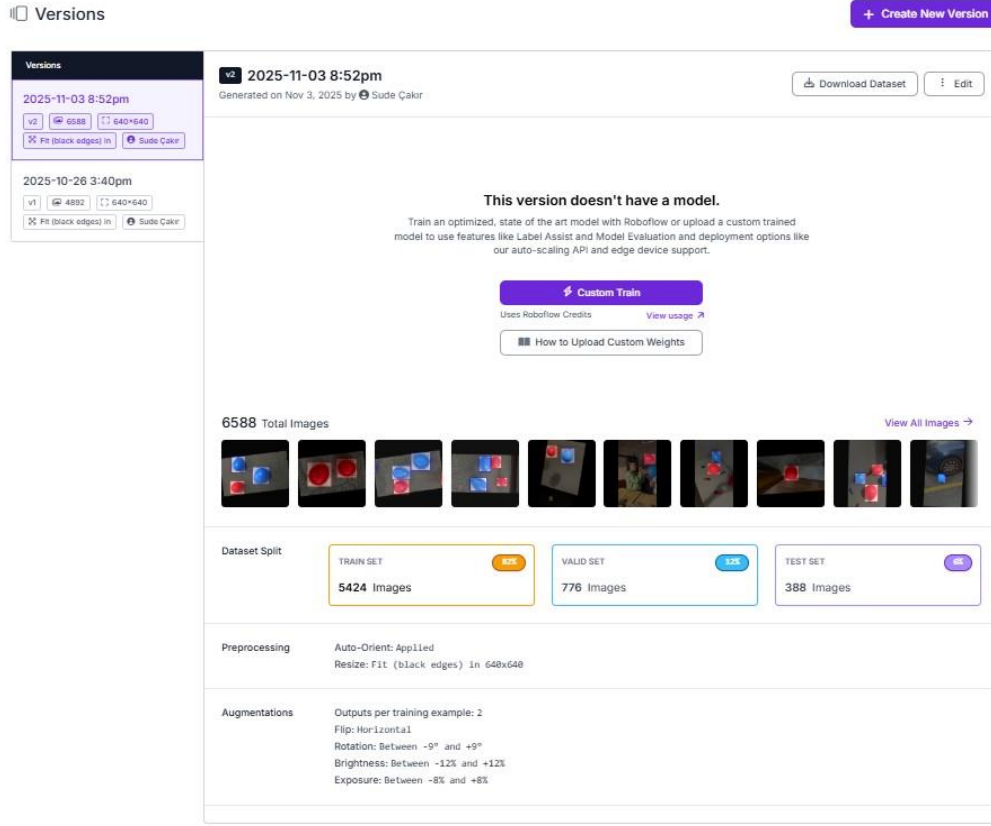
Modelin doğruluk seviyesini artırmak amacıyla veri setinin ikinci sürümü (V2) oluşturulmuştur. Bu sürümde, farklı arka planlarda, daha düşük ışık koşullarında ve daha geniş mekânsal varyasyon içeren **848 yeni görüntü** eklenmiş ve aynı şekilde manuel etiketleme işlemi yapılmıştır. Böylece V2 sürümünde toplam görüntü sayısı **6588'e** ulaşmış; eğitim seti 5424, doğrulama seti 776 ve test seti 388 görüntüden oluşacak şekilde güncellenmiştir. Eklenen bu yeni görüntüler, özellikle düşük kontrastlı sahneler ve kısmi balon görünürlüğü gibi zorlayıcı durumlarda model performansını artırmayı hedeflemiştir.

The screenshot shows the Roboflow V1 dataset version page. The page is titled 'Versions' and has a '+ Create New Version' button. The main content area shows the details for version V1, which was generated on Oct 26, 2025, by Sude Çakır. The dataset is split into three sets: Train Set (4026 images), Valid Set (577 images), and Test Set (289 images). The preprocessing settings are: Auto-Orient: Applied, Resize: Fit (black edges) in 640x640. The augmentations are: Outputs per training example: 2, Flip: Horizontal, Rotation: Between -9° and +9°, Brightness: Between -12% and +12%, Exposure: Between -8% and +8%.

Dataset Split	Train Set	Valid Set	Test Set
	4026 Images	577 Images	289 Images

Preprocessing	Augmentations
Auto-Orient: Applied Resize: Fit (black edges) in 640x640	Outputs per training example: 2 Flip: Horizontal Rotation: Between -9° and +9° Brightness: Between -12% and +12% Exposure: Between -8% and +8%

Şekil 3: V1 sürümüne ait Roboflow veri seti ekranı



Şekil 4: V2 sürümüne ait Roboflow veri seti ekranı

Bu iki şekil alt alta yerleştirildiğinde, veri setinin geliştirilme süreci görsel olarak da açık bir şekilde sunulmuş olur. Veri hazırlama aşaması, iş-zaman çizelgesinde belirtilen tarihler içinde tamamlanmış ve model eğitimi için yüksek çeşitlilik düzeyine sahip, dengeli ve özgün bir veri kümesi elde edilmiştir.

2.3.2 Yapay Zekâ Modelinin Eğitimi

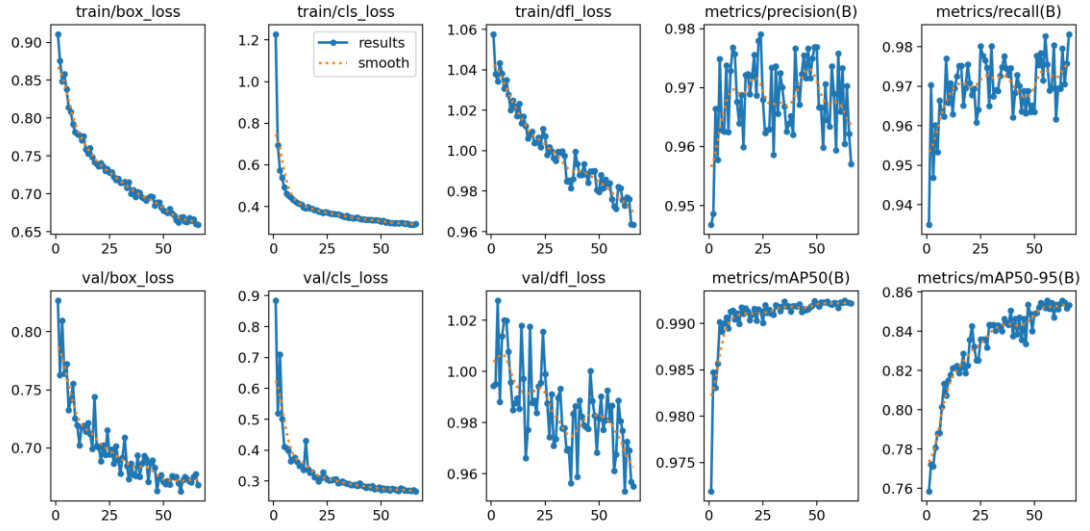
Proje kapsamında kullanılacak yapay zekâ tabanlı nesne tespit modeli olarak “You Only Look Once (YOLO)” mimarisi tercih edilmiştir. YOLO ailesi, gerçek zamanlı nesne tespiti için optimize edilmiş yapısı sayesinde hem gömülü sistemlerde hem de GPU tabanlı hızlı işlemede başarıyla kullanılan modern derin öğrenme modellerindendir. Tek aşamalı (single-stage) algılama yaklaşımı, YOLO modellerinin yüksek kare hızına (FPS) ulaşmasını ve düşük gecikme süresiyle çalışmasını sağlamaktadır. Projenin gerçek zamanlı hedef tespit, sınıflandırma ve

servo-lazer yönlendirme gereksinimleri göz önüne alındığında, YOLO'nun yapısı bu sistem için ideal bir mimari oluşturmaktadır.

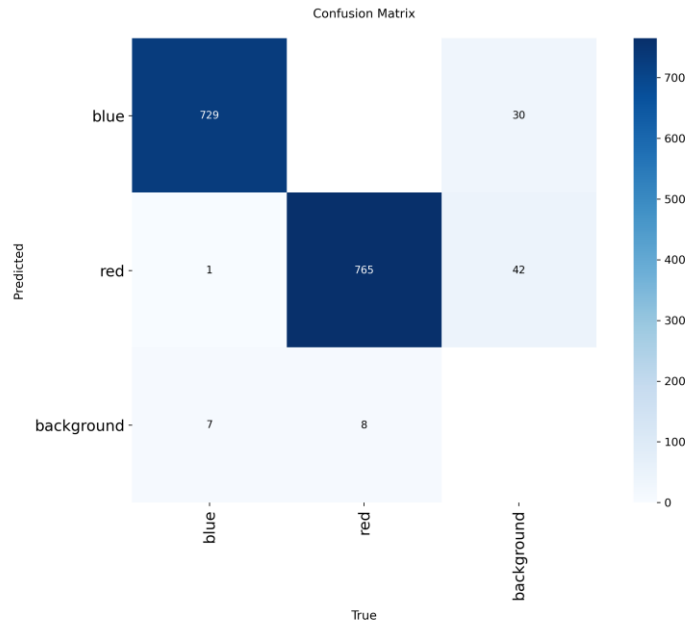
Bu çalışma özelinde, YOLO ailesinin en güncel sürümü olan **YOLOv8** kullanılmıştır. YOLOv8; gelişmiş anchor-free mimarisi, optimize edilmiş C2f blokları, hız-doğruluk dengesi, esnek giriş boyutları ve modern eğitim altyapısıyla önceki YOLO sürümlerine göre önemli iyileştirmeler sağlamaktadır. Ayrıca Ultralytics tarafından sağlanan güçlü kütüphane desteği, model eğitiminin Colab ortamında kolayca yürütülmesini ve eğitim sonuçlarının hızlıca analiz edilmesini mümkün kılmıştır. Bu avantajlar nedeniyle YOLOv8, proje kapsamında kullanılmak üzere en uygun model mimarisi olarak belirlenmiştir.

YOLOv8, farklı boyutlarda modeller (n, s, m, l, x) sunmakta olup model boyutları büyüdükçe doğruluk artmakta ancak hız ve gömülü cihazlarda çalışma performansı düşmektedir. Projenin hedefleri arasında sistemin Raspberry Pi 5 üzerinde gerçek zamanlı çalışması bulunduğundan, hem hızlı çıkarım süresi hem de yeterli doğruluk elde etmek amacıyla **YOLOv8n (nano)** modeli tercih edilmiştir. YOLOv8n modeli yaklaşık 3 milyon parametreye sahip olup düşük donanım gereksinimiyle yüksek FPS performansı sağlaması, gömülü platforma aktarım için büyük avantaj sunmaktadır. Ancak bu modelin daha küçük yapısı, büyük modellere göre nispeten daha düşük doğruluk potansiyeli barındırmaktadır. Bu dezavantaj, proje kapsamında oluşturulan geniş ve çeşitlendirilmiş veri seti ile dengelenmiş ve model performansı yüksek seviyelere çıkarılmıştır.

Model eğitimi Google Colab ortamında, YOLOv8n mimarisi kullanılarak gerçekleştirilmiştir. Eğitim sürecine ait kayıp fonksiyonları, doğruluk eğrileri ve diğer performans metrikleri Notebook üzerinde analiz edilmiştir. Eğitim süreci boyunca hem eğitim hem doğrulama kayıplarının aşamalı olarak azaldığı; precision, recall ve mAP metriklerinin yüksek seviyede seyrettiği gözlemlenmiştir. Confusion matrix incelendiğinde ise kırmızı ve mavi balon sınıflarının oldukça tutarlı biçimde ayrıldığı, yanlış pozitif ve yanlış negatif sayılarının düşük seviyelerde kaldığı görülmektedir. Bu sonuçlar, eğitilen modelin dost-düşman ayrımını sahne üzerinde güvenilir şekilde gerçekleştirebildiğini ortaya koymaktadır.



Şekil 5: YOLOv8 Eğitim Sürecine Ait Kayıp Fonksiyonu ve Performans Eğrileri



Şekil 6: Eğitilmiş YOLOv8 Modeline Ait Confusion Matrix Çıktısı

Modelin sınıflandırma performansını gösteren confusion matrix incelendiğinde, kırmızı ve mavi balonların yüksek doğrulukla tespit edildiği görülmektedir. Mavi balon sınıfında 729, kırmızı balonda ise 765 doğru sınıflandırma yapılmış olup yanlış pozitif ve yanlış negatif değerlerin oldukça düşük seviyede kalması modelin renk temelli dost-düşman ayrımını güvenilir şekilde gerçekleştirdiğini göstermektedir.

YOLOv8 modelinin eğitim süreci boyunca elde edilen kayıp ve performans grafiklerinde, hem eğitim hem de doğrulama kayıplarının düzenli olarak azaldığı, precision ve recall değerlerinin yüksek seviyelerde seyrettiği ve mAP skorlarının her epoch ile birlikte arttığı görülmektedir. Bu sonuçlar, modelin istikrarlı şekilde öğrenmeye devam ettiğini ve balon tespit performansının başarılı şekilde iyileştiğini göstermektedir.

```
Ultralytics 8.3.221 Python-3.12.12 torch-2.8.0+cu126 CUDA:0 (NVIDIA A100-SXM4-40GB, 40507MiB)
Model summary (fused): 72 layers, 3,006,038 parameters, 0 gradients, 8.1 GFLOPs
val: Fast image access (ping: 0.0±0.0 ms, read: 1145.4±321.3 MB/s, size: 37.9 KB)
val: Scanning /content/Red-Blue-Balloon-Detection-1/valid/labels.cache... 577 images, 0 backgrounds, 0 corrupt: 100% 577/577 1.2Mit/s 0.0s
Class Images Instances Box(P R mAP50 mAP50-95) 19/19 4.7it/s 4.0s
all 577 1510 0.963 0.975 0.992 0.855
blue 489 737 0.969 0.977 0.993 0.854
red 483 773 0.958 0.974 0.991 0.856
Speed: 1.3ms preprocess, 1.2ms inference, 0.0ms loss, 0.7ms postprocess per image
Results saved to /content/runs/detect/val

=== VALIDATION SONUÇLARI ===
Precision: 0.992
Recall: 0.855
mAP50: 0.992
mAP50-95: 0.973
```

Şekil 7: YOLOv8 validasyon performans sonuçları

Validasyon sonuçları incelendiğinde, YOLOv8 modelinin kırmızı ve mavi balonları yüksek doğrulukla tespit ettiği görülmektedir. Model genel olarak %0.992 precision, %0.855 recall, %0.992 mAP@50 ve %0.973 mAP@50–95 değerlerine ulaşmıştır. Sınıflar bazında da box, precision ve recall metriklerinin tutarlı olduğu, her iki sınıf için de mAP değerlerinin yüksek seviyede seyrettiği görülmektedir. Bu sonuçlar, oluşturulan veri setinin model tarafından başarıyla öğrenildiğini ve sistemin gerçek zamanlı testler için yeterli performansa sahip olduğunu göstermektedir.

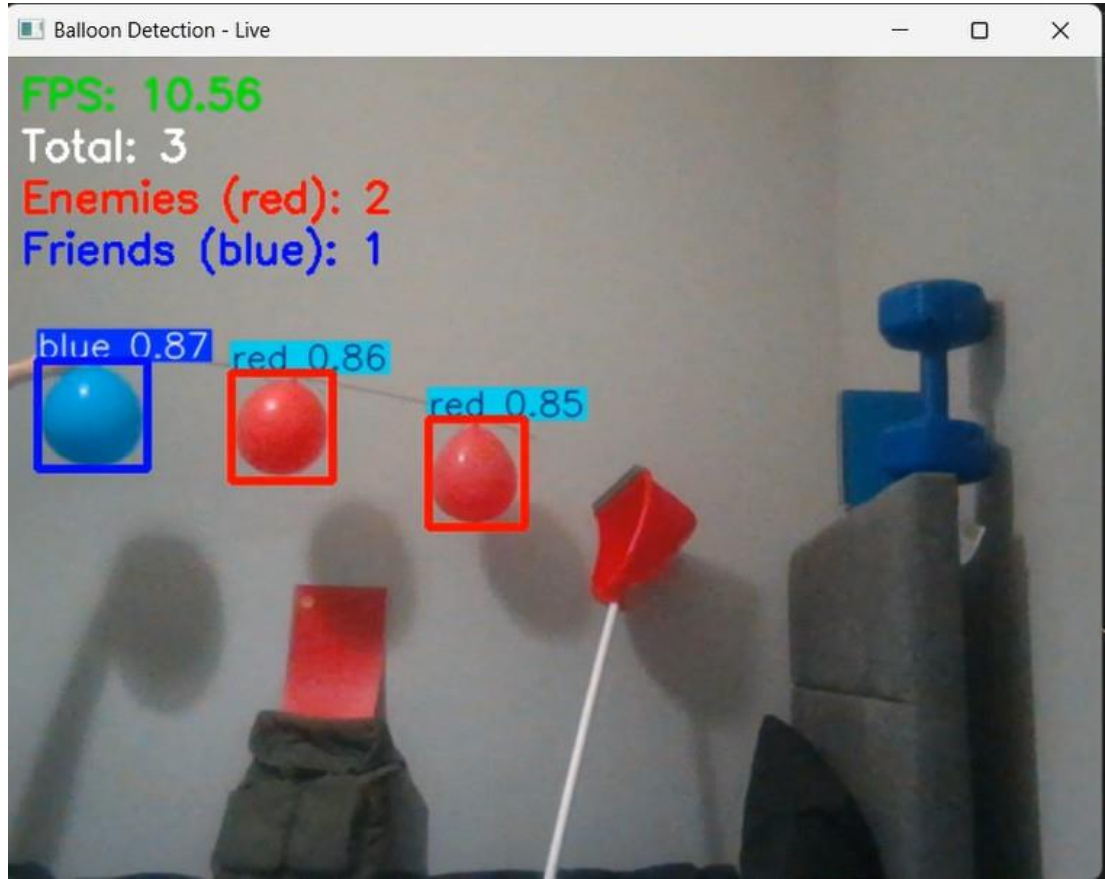
2.3.3 Bilgisayar Ortamında Model Testleri ve Tespit-İşaretleme Algoritmasının Oluşturulması

YOLOv8n modelinin eğitim sürecinin tamamlanmasının ardından, modelin sahadaki davranışını değerlendirmek amacıyla bilgisayar ortamında kapsamlı testler gerçekleştirilmiştir. Bu testler, hem kayıtlı videolar hem de gerçek zamanlı kamera akışı üzerinde uygulanmış; ardından servo–lazer yönlendirme aşamasında kullanılacak olan tespit-işaretleme algoritmasının (Lock–Impact–Kill) ilk prototipi geliştirilmiştir. Böylece, gömülü sistem tarafına geçmeden önce modelin performansı, kararlılığı ve sistem mimarisine uyumu doğrulanmıştır

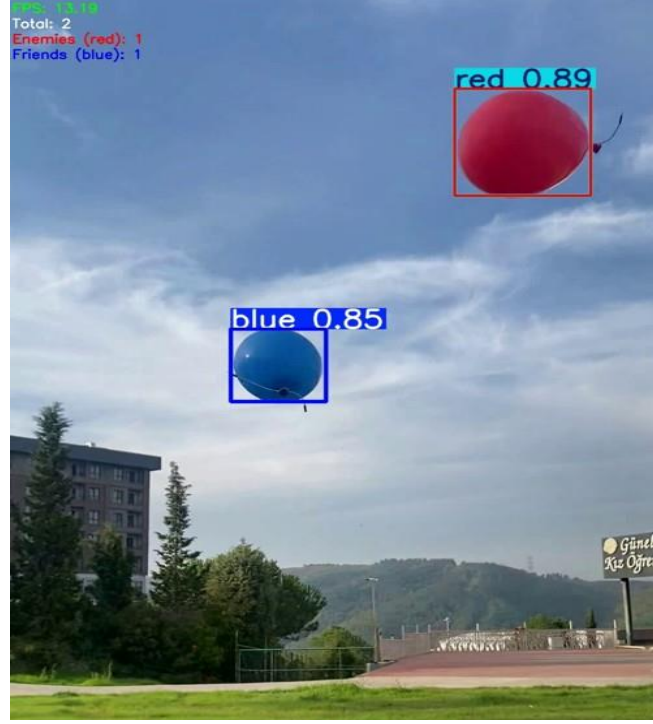
2.3.3.1 Bilgisayar Ortamında Tespit Testleri (Video + Realtime)

Model ilk olarak farklı ışık koşullarına, açı değişimlerine ve arka plan farklılıklarına sahip kayıtlı videolar üzerinde test edilmiştir. Bu testler, modelin gerçek sahne koşullarına uyum kabiliyetini değerlendirmek amacıyla yapılmıştır. Elde edilen çıktılarda kırmızı balonların düşman, mavi balonların dost olarak doğru sınıflandırıldığı; tespit kutularının hedefi kararlı biçimde takip ettiği görülmüştür.

Video testlerinin ardından model, bilgisayar ortamında gerçek zamanlı kamera akışı üzerinde de sınanmıştır. Bu aşamada sistemin kare başına işlem süresi (FPS), gecikme süresi ve hızlı hareketlere verdiği tepki ölçülmüştür. Gerçek zamanlı testlerde model ortalama **10-15 FPS** aralığında çalışmış; dost-düşman ayrımı canlı görüntüde yüksek doğrulukla gerçekleştirilmiştir. Böylece YOLOv8n modelinin gömülü sisteme aktarım için yeterli performans düzeyine ulaştığı doğrulanmıştır.



Şekil 8: Gerçek Zamanlı Webcam Çıktısı



Şekil 9: Video testlerinden örnek kareler

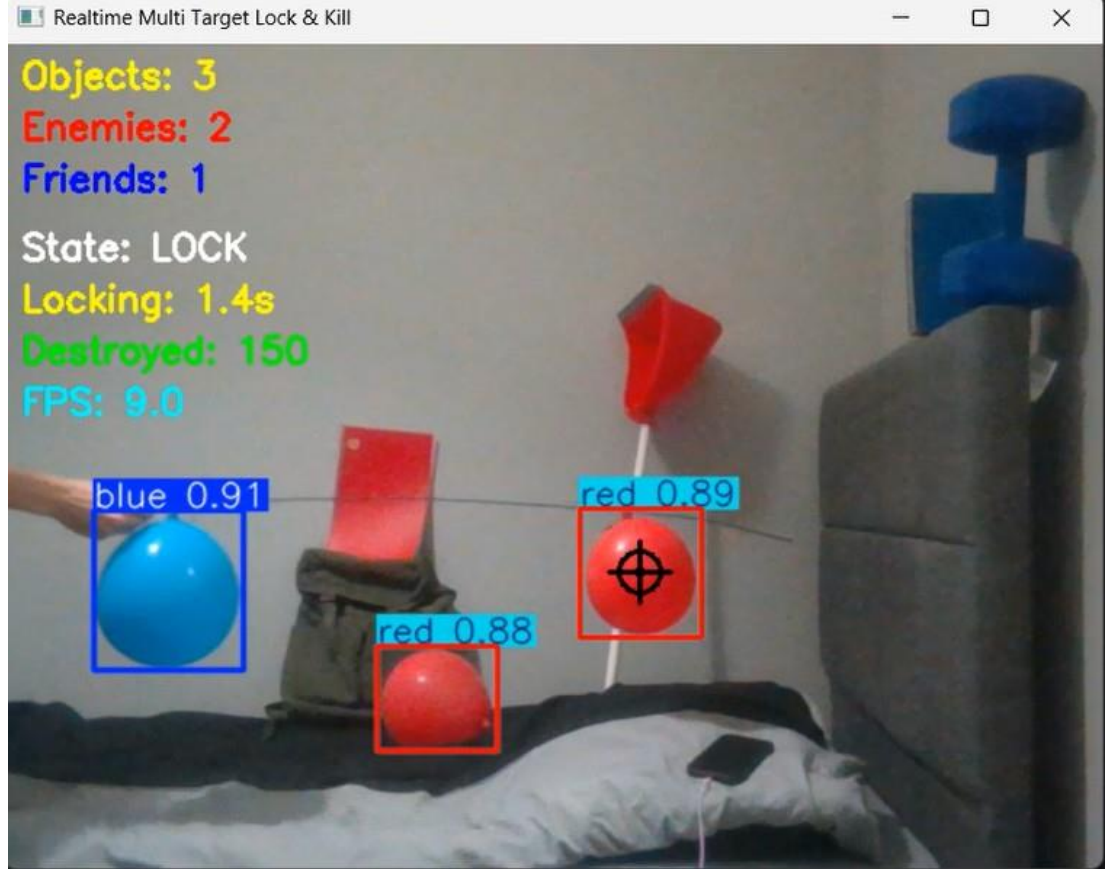
2.3.3.2 Tespit-İşaretleme (Lock-İmpact-Kill) Algoritmasının Bilgisayar Ortamında Geliştirilmesi

Bilgisayar ortamındaki tespit testlerinin ardından, düşman hedefe odaklanma ve işaretleme mekanizmasını oluşturan Lock-İmpact-Kill algoritması geliştirilmiştir. Bu algoritma, ilerleyen aşamalarda servo-motor ve lazer modülüyle bütünleşecek olan otonom hedefleme sisteminin temelini oluşturmaktadır. Sistemin daha kontrollü ve kararlı çalışması için bu yapı aynı zamanda basit bir FSM (Finite State Machine) yaklaşımıyla tasarlanmıştır. FSM yapısı, sistemin her anda yalnızca tek bir durumda (IDLE, LOCK veya KILL) bulunmasını sağlayarak, durum geçişlerinin daha düzenli ve hata toleransının yüksek olmasına imkân tanımaktadır.

Algoritmanın üç ana aşaması bulunmaktadır:

1. **Lock (Kilitleme):** Kırmızı (düşman) hedef belirli bir süre sürekli algılanırsa kilitlemiş kabul edilir. Hedefin konumu kareden kareye takip edilir ve odak noktası belirlenir.
2. **Impact (Etkileşim):** Kilit süresi tamamlandığında sistem hedefin merkezine “nişangâh” sembolünü yerleştirir. Bilgisayar ortamında bu işlem görsel olarak bir işaretleme ile gösterilmiştir.
3. **Kill (İmha Kaydı):** İşaretlenen hedef “imha edildi” olarak işlenir.

Bu yapı, Raspberry Pi üzerinde servo yönlendirme ve lazer aktivasyonu için kullanılacak olan kontrol döngüsünün temel davranışını simüle etmektedir. Aynı anda birden fazla düşman hedef bulunduğunda algoritma hedefleri önceliklendirme mekanizması ile sırayla işlemekte ve her bir hedefin imha sürecini bağımsız olarak kaydetmektedir.



Şekil 10: Lock–Impact–Kill algoritmasının bilgisayar ortamında real-time testi

2.3.4 Raspberry Pi’da Sistem Kurulumu ve YOLO Algoritmasının Uygulanması

Eğitilen YOLOv8n modelinin bilgisayar ortamında elde edilen başarılı sonuçlarının ardından model, gömülü platform üzerinde gerçek zamanlı olarak çalıştırılmak üzere **Raspberry Pi 5** cihazına aktarılmıştır. Projenin bu aşamasında amaç; düşük güç tüketimli, kompakt ve taşınabilir bir donanım üzerinde yüksek FPS değerleriyle nesne tespiti yapabilen bir altyapı oluşturmaktır. Bu doğrultuda Raspberry Pi 5, sahip olduğu geliştirilmiş işlemci mimarisi, artan bellek kapasitesi ve kamera arayüzü desteği nedeniyle proje için en uygun gömülü sistem platformu olarak tercih edilmiştir.

Raspberry Pi 5; dört çekirdekli 64-bit ARM Cortex-A76 işlemcisi, 2,4 GHz'e kadar çıkan saat frekansı, PCIe 2.0 destekli yüksek hızlı bağlantı mimarisi, gelişmiş termal performansı ve Camera Module 3 ile tam uyumu sayesinde, gerçek zamanlı görüntü işleme ve yapay zeka uygulamaları için önceki Pi modellerine kıyasla önemli hız kazanımları sunmaktadır. Pi 5'in artırılmış bellek bant genişliği ve güçlü GPU/VPU birimleri, YOLO gibi hesaplama yoğun modellerin çalıştırılmasında belirgin performans avantajı sağlayarak bu projede tercih edilmesinin en önemli gerekçeleri olmuştur.



Şekil 11: Raspberry Pi 5

Modelin gömülü sisteme aktarılması sürecinde ilk olarak cihaz üzerinde yazılım ortamı hazırlanmıştır. Raspberry Pi OS kurulmuş, Python tabanlı çalışma ortamı yapılandırılmış ve OpenCV, NumPy, PyTorch (ARM destekli), Ultralytics YOLO kütüphanesi ve kamera yönetimi için gerekli paketler yüklenmiştir. Kamera modülü bağlantısı doğrulandıktan sonra libcamera ve v4l2 altyapıları test edilerek donanımın sorunsuz çalıştığı doğrulanmıştır.

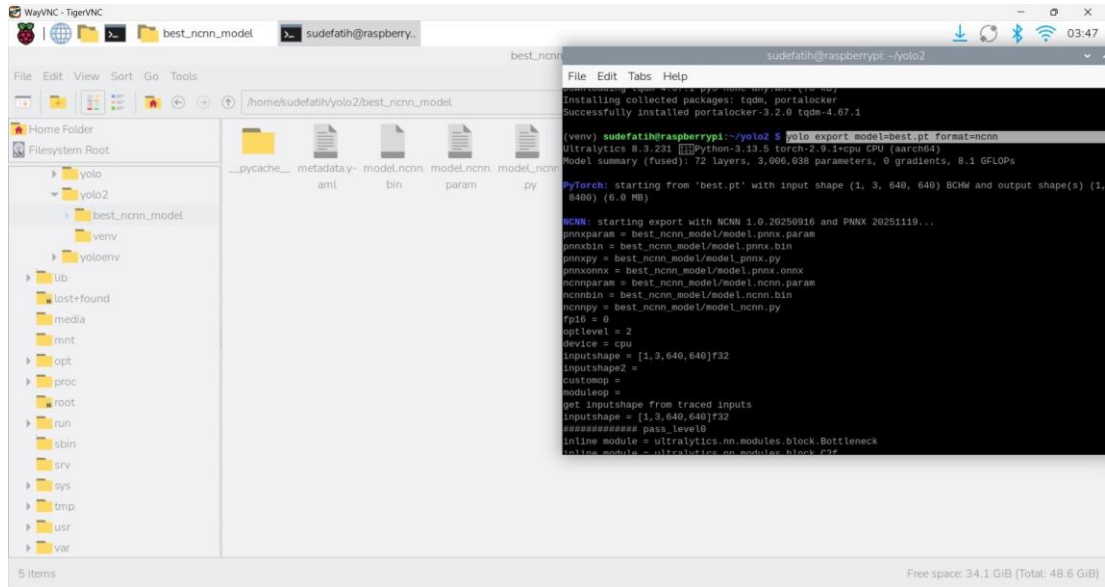
Eğitilen model dosyası (*best.pt*), bilgisayar ortamından Raspberry Pi 5'e aktarılmış ve başlangıç testleri doğrudan Ultralytics üzerinden yapılmıştır. Ancak Pi üzerinde daha yüksek performans elde edebilmek amacıyla modelin **NCNN formatına dönüştürülmesine** karar verilmiştir. NCNN; Tencent tarafından geliştirilen, ARM

mimarisi için optimize edilmiş, GPU bağımsız çalışan yüksek hızlı bir inference kütüphanesidir. YOLO modellerinin gömülü sistemlerde FPS artışı için sıklıkla tercih edilmektedir.

Bu doğrultuda model dönüştürme süreci şu adımlarla gerçekleştirilmiştir:

1. **YOLOv8 → ONNX dönüştürme:** Ultralytics komutlarıyla eğitimden alınan *best.pt*, **ONNX** formatına aktarıldı.
2. **ONNX → NCNN dönüştürme:** **onnx2ncnn** ve **ncnnoptimize** araçları kullanılarak model optimize hale getirildi. Elde edilen dosyalar; **model.ncnn.param** ve **model.ncnn.bin**.
3. **Python NCNN YOLO inference testleri:** Raspberry Pi üzerinde görüntü işlemek için NCNN tabanlı YOLO inference kodu çalıştırıldı.

Bu dönüştürme sayesinde Raspberry Pi 5 üzerinde çıkarım süresi (inference time) önemli ölçüde azalmış, model daha yüksek FPS değerlerine ulaşmıştır. Bu aşamada hem tek kare inference testleri hem de kısa video akışları üzerinde hız ölçümleri yapılmış; modelin hem doğruluk hem hız açısından gömülü platformda çalışmaya hazır olduğu doğrulanmıştır.



Şekil 12: ONNX → NCNN dönüşüm aşamalarını gösteren terminal çıktıları

Modelin Raspberry Pi 5 üzerinde çalıştırılması sonrasında, gömülü platformda elde edilen çıkarım (inference) performansının değerlendirilmesi amacıyla kayıtlı bir video dosyası üzerinde farklı çözünürlüklerde FPS ölçümleri yapılmıştır. Bu testler sayesinde hem NCNN optimizasyonunun sağladığı performans artışı incelenmiş hem

de sistem için en uygun çözünürlük belirlenmiştir. Her çözünürlükte inference süresi ve ortalama FPS değerleri ayrı ayrı hesaplanmış, elde edilen sonuçlar aşağıdaki gibidir:

- **640×360 çözünürlük:** 9.24 FPS
- **360×640 çözünürlük:** 8.52 FPS
- **854×480 çözünürlük:** 8.55 FPS
- **480×854 çözünürlük:** 8.60 FPS
- **1280×720 çözünürlük:** 7.45 FPS
- **720×1280 çözünürlük:** 7.33 FPS

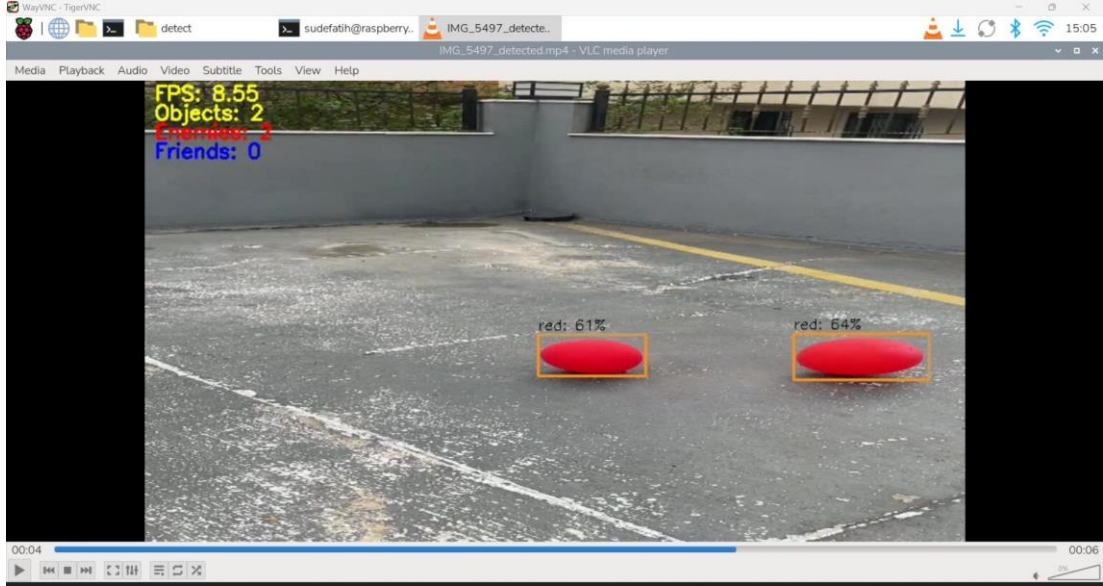
Elde edilen veriler, çözünürlük azaldıkça FPS değerinin genel olarak arttığını göstermektedir. Ancak 640×360 gibi düşük çözünürlüklerde, 1080p kaynak görüntüye kıyasla detay kaybı belirgin hâle gelmekte; küçük nesnelerin görünürlüğü ve tespit kararlılığı azalmaktadır. Bu nedenle rapor kapsamında kullanılacak örnek çıktı görseli ve sistemin sonraki aşamalarında kullanılacak standart çözünürlük olarak **854×480** seçilmiştir. Bu çözünürlük, sahnedeki detayları korurken (1080p'nin yarısından fazla detay tutularak), aynı zamanda Raspberry Pi üzerinde **8.55 FPS** ile yeterli gerçek zamanlı performans sağlamaktadır.

Ayrıca 854×480 çözünürlüğünde elde edilen 8.55 FPS, YOLOv8n modelinin NCNN altyapısı üzerinde optimize edilmesiyle Raspberry Pi 5'ten alınabilecek performansın tatmin edici seviyede olduğunu göstermektedir. Bu değerler, proje kapsamında ileride uygulanacak servo-lazer yönlendirme ve gömülü tespit-takip mekanizmalarının gerçek zamanlı şekilde çalışması için yeterli altyapıyı sunmaktadır.

İlerleyen aşamalarda FPS değerinin daha da artırılması için uygulanabilecek geliştirme adımları ise şu şekildedir:

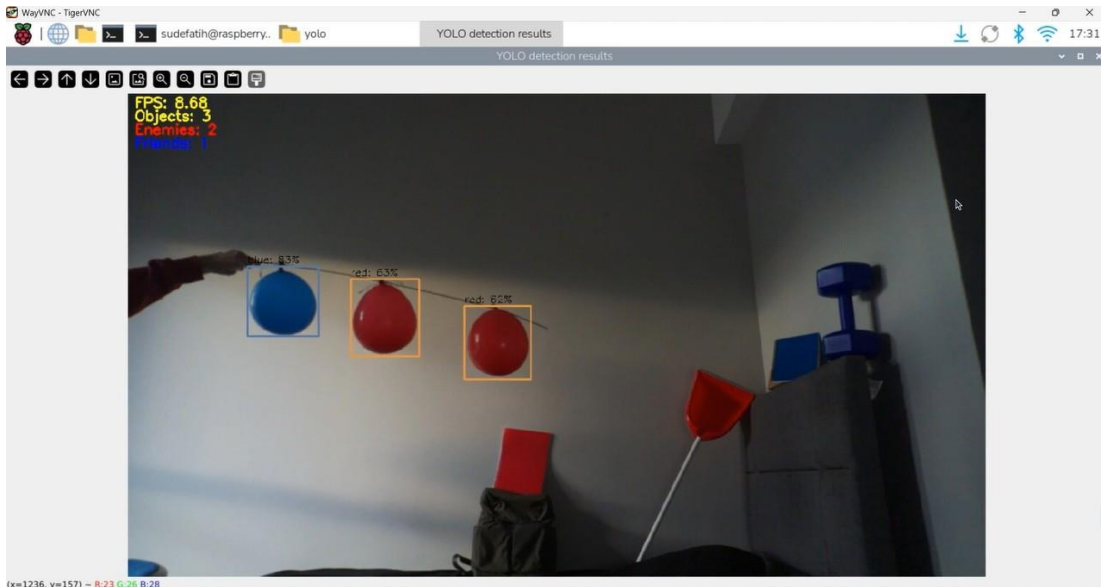
- NCNN üzerinde FP16 veya INT8 quantization yapılması.
- Tile-based (patch) inference yöntemlerinin denenmesi.
- Raspberry Pi GPU hızlandırması için Vulkan backend desteğinin etkinleştirilmesi.
- Raspberry Pi için özel optimize edilmiş lightweight backbone modellerinin (MobileNet-YOLO, YOLOv8n-Slim vb.) test edilmesi.

Bu iyileştirmeler ışığında sistemin gelecekte daha yüksek FPS seviyelerine ulaşması mümkündür. Bu bölümde sunulan örnek çıktı ise, 854×480 çözünürlüğünde çalışan NCNN tabanlı YOLOv8n modelinin Raspberry Pi 5 üzerinde elde ettiği tespit performansını göstermekte olup Şekil X'te verilmiştir..



Şekil 13: Raspberry Pi 5 üzerinde NCNN tabanlı YOLOv8n modelinin 854×480 çözünürlüğünde tespit performansı ve FPS çıktısı.

Erişim linki: [Link](#)



Şekil 14: Raspberry Pi 5 Kamera ile NCNN tabanlı YOLOv8n modelinin 854×480 çözünürlüğünde tespit performansı ve FPS çıktısı.

Erişim linki: [Link](#)

2.3.5 Raspberry Pi da Gerçek Zamanlı Tespit-İşaretleme Algoritmalarının Uygulanması

Eğitilen YOLOv8n modelinin Raspberry Pi 5 üzerinde gerçek zamanlı olarak çalıştırılmasıyla, dost-düşman ayrımı yapan tespit, hedefe kilitlenme (lock) ve işaretleme-îmhâ (impact) aşamalarını içeren **tamamen otonom bir tespit-takip algoritması** başarıyla uygulanmıştır. Bu aşama, projenin bütünleşik yapay zekâ + FSM (Finite State Machine) tabanlı karar verme sisteminin gömülü platformda bütünleştiği kritik bölümüdür.

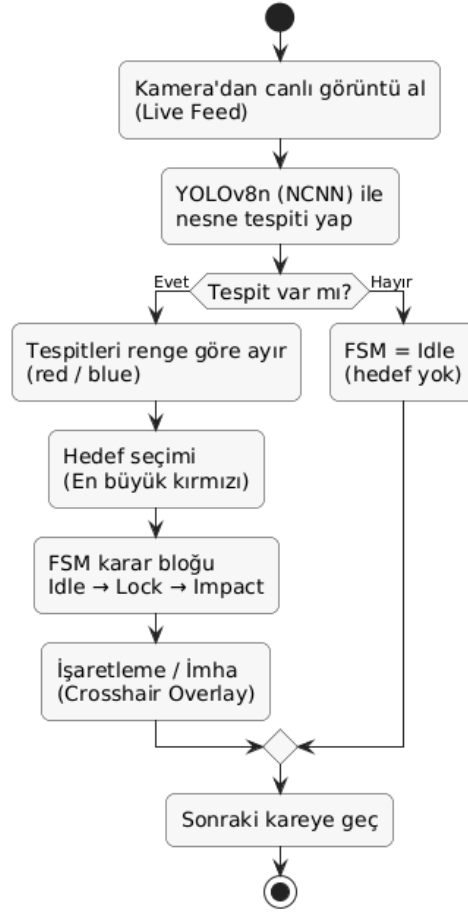
Raspberry Pi 5'in geliştirilmiş ARM Cortex-A76 mimarisi, PCIe tabanlı depolama hızı ve iyileştirilmiş görüntü işleme kapasitesi sayesinde YOLOv8n modelinin NCNN optimizasyonlu hâli ile **8–10 FPS** arası kararlı gerçek zamanlı çıkarım performansı elde edilmiştir. Bu FPS değerleri, servo-lazer modül entegrasyonu için gerekli olan zamanlama gereksinimlerini karşılamakta ve sistemin gerçek sahne üzerinde akıcı biçimde çalışmasına imkân sağlamaktadır.

2.3.5.1 Gerçek Zamanlı Tespit Akışının Genel Mimarisi

Raspberry Pi üzerinde çalışan gerçek zamanlı tespit-işaretleme sistemi, dört ana bileşenden oluşmaktadır:

1. Görüntü Toplama Modülü (Picamera2)
2. YOLOv8n-NCNN Inference Motoru
3. Finite State Machine (FSM) Tabanlı Hedef Yönetimi
4. İşaretleme ve İmha Simülasyonu (Lock-Impact)

Raspberry Pi Gerçek Zamanlı Tespit-İşaretleme Algoritması (Activity Diagram)



Şekil 15: Raspberry Pi Gerçek Zamanlı Tespit-İşaretleme Algoritması Akış Şeması

2.3.5.2 Finite State Machine (FSM) Tabanlı İşaretleme Sistemi

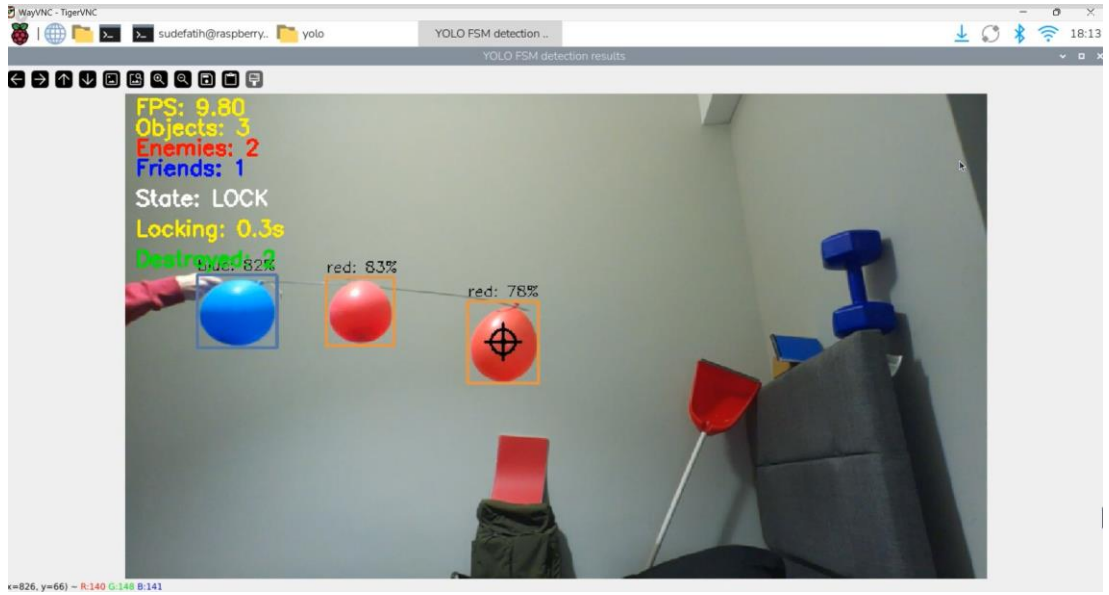
Gerçek zamanlı tespit aşamasının en önemli bileşeni, **hedefin belirlenmesi, takip edilmesi ve işaretleme**ni yöneten finite state machine yapısıdır. FSM, aşağıdaki üç temel durumdan oluşur:

1. IDLE → Kırmızı hedef yok → sistem bekleme konumunda
2. LOCK → Hedef tespit edildi → hedef merkezde tutulur → kilit süresi başlar
3. IMPACT → Kilit süresi tamamlandı, imha simülasyonu uygulanır, kayıt yapılır.

2.3.5.3 Gerçek Zamanlı Tespit ve İşaretleme Çıktıları

Raspberry Pi üzerinde doğrudan kamera akışı üzerinden yapılan testlerde, sistemin dost (blue) ve düşman (red) hedefleri ayrıştırdığı, düşman hedefe kilitlenip belirlenen sürede imha ettiği gözlemlenmiştir.

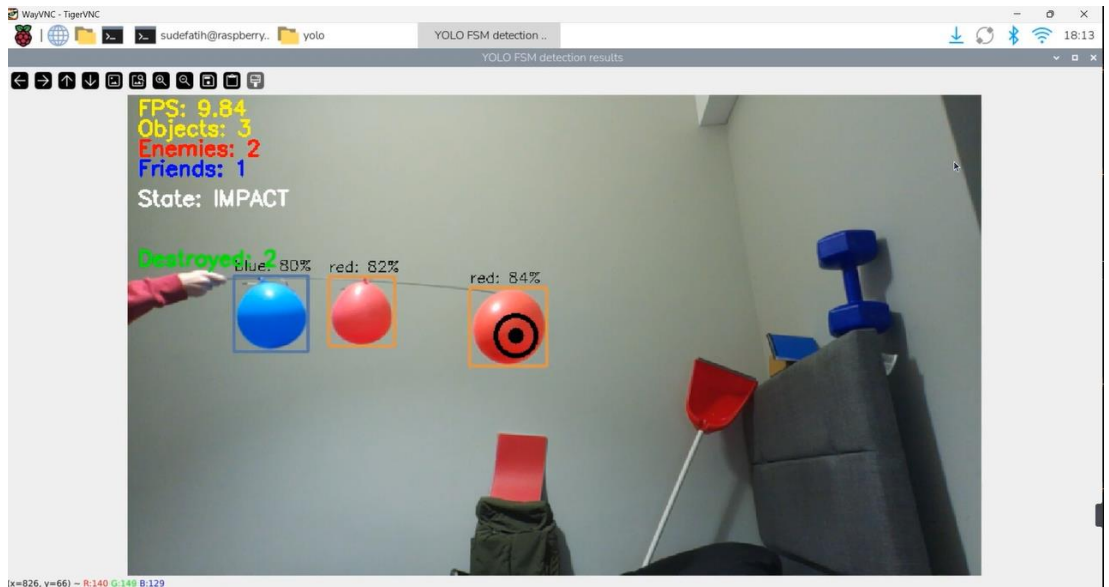
Aşağıdaki ekran görüntüleri gerçek zamanlı çalışmayı göstermektedir:



Şekil 16: Raspberry Pi üzerinde gerçek zamanlı tespit – LOCK durumu

Erişim Linki: [Link](#)

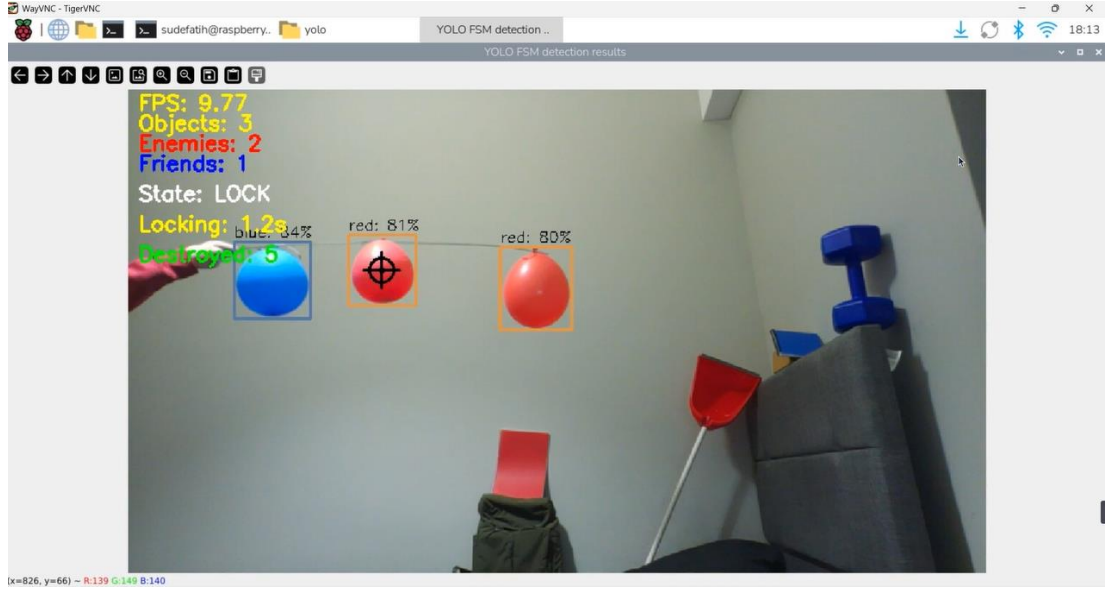
Bu aşamada; sistem en büyük kırmızı hedefe kilitlenmiştir. LOCK süresi ekranda geri saymaktadır. Dost hedef (mavi) doğru şekilde ayrılmıştır. FPS değeri ~9 civarındadır



Şekil 17: Raspberry Pi üzerinde işaretleme – IMPACT durumu

Erişim Linki: [Link](#)

Bu aşamada; Kilitlenen hedef başarıyla “işaretlenmiş/etkisiz hale getirilmiştir”. Crosshair simülasyonu genişleyerek imha efektini göstermektedir. Dost hedef (mavi) doğru şekilde ayrılmıştır. Destroyed counter artmaktadır.



Şekil 18: Gerçek zamanlı çalışmada birden fazla hedefin yönetimi

Erişim Linki: [Link](#)

Bu senaryoda, FSM bir önceki imhâ edilen hedeflerin koordinatlarını bellekte tutarak tekrar hedef alınmalarını **cooldown mekanizmasıyla** engellemiştir. Böylece sistem aynı hedef üzerinde tekrar tekrar tetiklenmek yerine yeni tehditlere yönelmiştir.

2.3.5.4 Raspberry Pi Üzerinde Performans (FPS – Gecikme) Analizi

Raspberry Pi 5 üzerinde yapılan gerçek zamanlı çalışma sırasında ortalama performans:

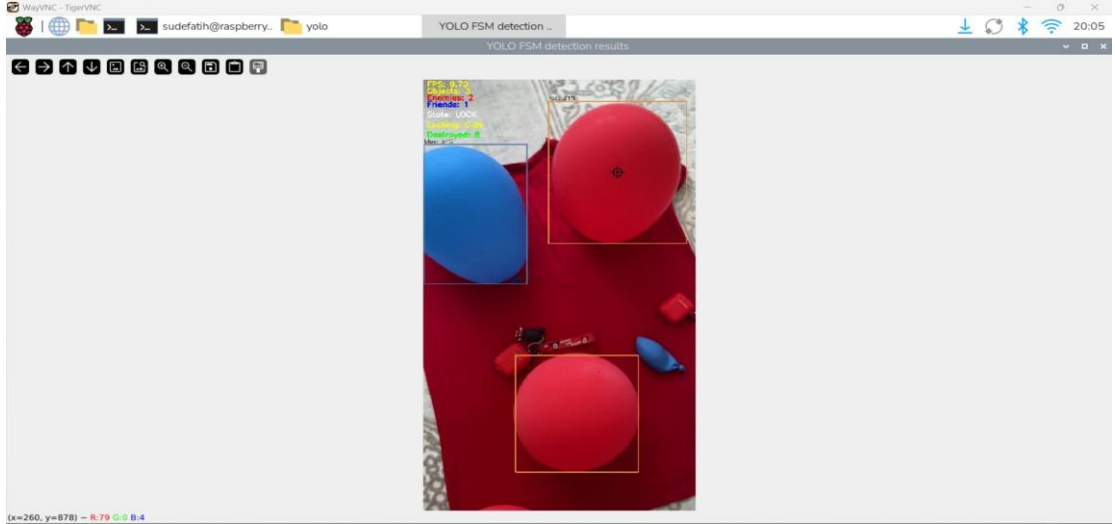
- **FPS: 8.5 – 9.8**
- **Inference Latency: ~100–120 ms**
- **Çözünürlük: 854×480, 1280x720**

Bu değerler servo–lazer modülü ile gerçek zamanlı hedef merkezleme için yeterli seviyededir.



Şekil 19: Raspberry Pi Üzerinde Performans Analiz Görüntüsü 1

Erişim Linki: [Link](#)



Şekil 20: Raspberry Pi Üzerinde Performans Analiz Görüntüsü 2

Erişim Linki: [Link](#)

Sonuç olarak, gerçek zamanlı çalışmalarda FPS, latency ve tespit kararlılığı farklı senaryolarda test edilmiş; sistemin sonraki aşamalarında yapılacak optimizasyonlar için bu veriler kayıt altına alınmıştır.

2.3.5.5 Gerçek Zamanlı Destroy Log (CSV) Kaydı

Sistem her imha edilen hedef için otomatik olarak aşağıdaki bilgileri kaydetmektedir:

- Hedef ID
- İmha zamanı (timestamp)
- İmha edilen hedefin merkez koordinatı

	A	B	C	D	E	F	G	H	I	J	K	L
1	target_id,destroy_time,center_x,center_y											
2	3,2025-11-23 19:30:06,331,350											
3	4,2025-11-23 19:30:10,565,183											
4	5,2025-11-23 19:30:13,231,97											
5	6,2025-11-23 19:30:17,224,361											
6	7,2025-11-23 19:30:21,345,130											
7	8,2025-11-23 19:30:25,224,362											
8	10,2025-11-23 19:30:29,530,257											
9	11,2025-11-23 19:30:33,223,361											
10	12,2025-11-23 19:30:37,435,320											
11												
12												
13												
14												
15												
16												
17												

Şekil 21: destroy_log.csv dosyası

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	Target ID	Date	Time	Center X	Center Y	Horizontal Pos	Vertical Pos								
2	3	2025-11-23	19:30:06	331	350	Center	Bottom								
3	4	2025-11-23	19:30:10	565	183	Right	Middle								
4	5	2025-11-23	19:30:13	231	97	Center	Top								
5	6	2025-11-23	19:30:17	224	361	Center	Bottom								
6	7	2025-11-23	19:30:21	345	130	Center	Top								
7	8	2025-11-23	19:30:25	224	362	Center	Bottom								
8	10	2025-11-23	19:30:29	530	257	Right	Middle								
9	11	2025-11-23	19:30:33	223	361	Center	Bottom								
10	12	2025-11-23	19:30:37	435	320	Right	Bottom								
11															
12															
13															
14															
15															
16															
17															
18															
19															
20															
21															

Şekil 22: destroy_log.xml dosyası

Bu kayıt yapısı, bir sonraki aşamada servo–lazer modülünün gerçek donanımsal tetikleme senaryosu ile uyumludur.

Bu rapor dönemine kadar gerçekleştirilen tüm iş paketleri iş-zaman çizelgesine uygun şekilde tamamlanmış olup, tespit–kilitleme–etki mekanizmasının hem bilgisayar ortamında hem de Raspberry Pi üzerinde çalışır hale getirilmesi sağlanmıştır. Kalan servo–lazer entegrasyonu, IoT/MQTT haberleşmesi ve tam sistem testleri final raporu döneminde gerçekleştirilecektir.

3. KAYNAKLAR

- Ultralytics YOLOv8 Resmi Dokümantasyonu
- <https://docs.ultralytics.com/>
- Glenn Jocher, Ultralytics – YOLOv8 GitHub Deposusu
- <https://github.com/ultralytics/ultralytics>
- NCNN (Tencent) – Neural Network Inference Framework Resmi Kaynakları
- <https://github.com/Tencent/ncnn>
- ONNX (Open Neural Network Exchange) Formatı
- <https://onnx.ai/>
- PNNX – PyTorch to NCNN Dönüştürücüsü
- <https://github.com/Tencent/ncnn/tree/master/tools/pnnx>
- Raspberry Pi 5 Resmi Teknik Dokümanı
- <https://www.raspberrypi.com/products/raspberry-pi-5/>
- Picamera2 Resmi Dokümantasyonu (Raspberry Pi Kamerası)
- <https://datasheets.raspberrypi.com/camera/picamera2-manual.pdf>
- JetBrains Research – State Machine (FSM) Tasarım Prensipleri
- Erişim: <https://www.jetbrains.com/help/>
- Embedding AI on Edge Devices – Edge AI Optimization Guide
- <https://www.edge-ai.org/>